Davante

MEDAC

Entornos de Desarrollo

Tema 1 - El Software

ENTORNOS DE DESARROLLO

El software

01

Programa informático

02

¿Qué es un lenguaje de programación?

03

Paradigmas de programación

04

Fases del proceso de compilación

Objetivos de la presentación

Concepto de programa informático

Comprender qué es y cómo funciona un programa.

Identificar los lenguajes de programación

Conocer las características y tipos de lenguajes disponibles

Conocer los paradigmas de programación

Conocer los diferentes enfoques para escribir programas

Entender la compilación

Conocer las fases por las que pasa un programa antes de ser ejecutado



/ 1. Programa informático

> Podemos definir un programa informático como una fórmula detallada de cómo resolver un problema determinado.

No puede ser ambiguo

Las instrucciones deben ser claras y precisas

Secuencia completa

Debe incluir todas las instrucciones necesarias para llegar a la solución

Orden correcto

Las instrucciones deben estar ordenadas adecuadamente.

Hay que diferenciar entre **programa informático** y **algoritmo**, pues este último permite indicar todos los pasos necesarios para poder resolver un problema que no tiene por qué ser informático. Podemos decir que un programa informático está compuesto por uno o diferentes algoritmos para poder resolver diferentes problemas, como, por ejemplo, la suma de dos números o encontrar el elemento más grande en un conjunto.

Software vs Programa informático

Un software puede considerarse como un conjunto de programas informáticos que se combinan entre sí para un determinado fin. Software puede ser desde un navegador, un sistema operativo, o un videojuego.

Para poder ejecutar un programa informático es necesario obtener un código que pueda ser comprendido por el procesador de la máquina. Por esta razón, cuando hablamos de programa informático, es necesario hacer referencia tanto al hardware como al software.



Navegador



Sistema Operativo



Videojuego

Hardware y software en los equipos

Cuando un programa informático va a ser ejecutado, el sistema operativo debe realizar la carga del programa, es decir, de sus datos e instrucciones en memoria RAM. Una vez que el programa se encuentra en memoria principal (RAM), es posible realizar su carga en la CPU.

Metáfora del carpintero

Disco Duro - Almacén

Todas las herramientas del carpintero están guardadas aquí

Memoria RAM - Mesa

El carpintero coloca las herramientas que va a usar sobre la mesa

CPU - Manos

El carpintero sostiene las herramientas más inmediatas en sus manos



Arquitectura Von Neumann

Todo sistema computacional actual sigue la arquitectura Von Neumann, con posibles modificaciones. Esta arquitectura especifica que todo computador requiere de una memoria RAM, un sistema de entrada/salida y una CPU.

La CPU estaría formada por registros, una unidad aritmético - lógica conocida como ALU y la unidad de control.

Registros

Pequeñas áreas de almacenamiento interno muy rápidas para guardar temporalmente datos e instrucciones

ALU

Unidad Aritmético-Lógica que realiza operaciones matemáticas y lógicas

Unidad de Control

Como el director de orquesta: indica al resto de componentes qué deben hacer y en qué orden



/ 2. ¿Qué es un lenguaje de programación?

Igual que existen idiomas entre humanos, un lenguaje de programación está formado por un conjunto de sentencias y expresiones que permiten que el programador se comunique con la máquina de una manera sencilla.

Lenguajes de programación y nivel de abstracción

Los lenguajes de programación actuales son lenguajes de alto nivel, entendiendo por ello el nivel de abstracción que poseen. El nivel de abstracción es la capacidad que tiene un individuo para obviar los detalles y aún así poder realizar las tareas encomendadas.

- Cuando un lenguaje tiene alto nivel de abstracción, oculta al programador detalles como el conjunto de instrucciones máquina disponibles en la CPU. Sin embargo, no se pueden ejecutar directamente sin un proceso de traducción conocido como compilación que convierte el código de alto nivel a lenguaje máquina, es decir, a bits, 1 y 0.



Lenguaje Máquina

El único lenguaje que la CPU entiende

1. **Formado exclusivamente por ceros y unos (bits)**

   Cada bit representa información binaria que el procesador puede interpretar

2. **Cada instrucción indica una acción concreta**

   Sumar, mover datos, comparar, saltar a otra instrucción, etc.

3. **Dependiente del tipo de procesador**

   Un código máquina para Intel no funciona directamente en ARM

Cómo funciona un lenguaje de alto nivel

1. **Código legible**

   Escribes instrucciones legibles por humanos (Python, Java, etc.)

2. **Bytecode**

   Algunos lenguajes generan un lenguaje intermedio ejecutable por una máquina virtual

3. **Máquina Virtual**

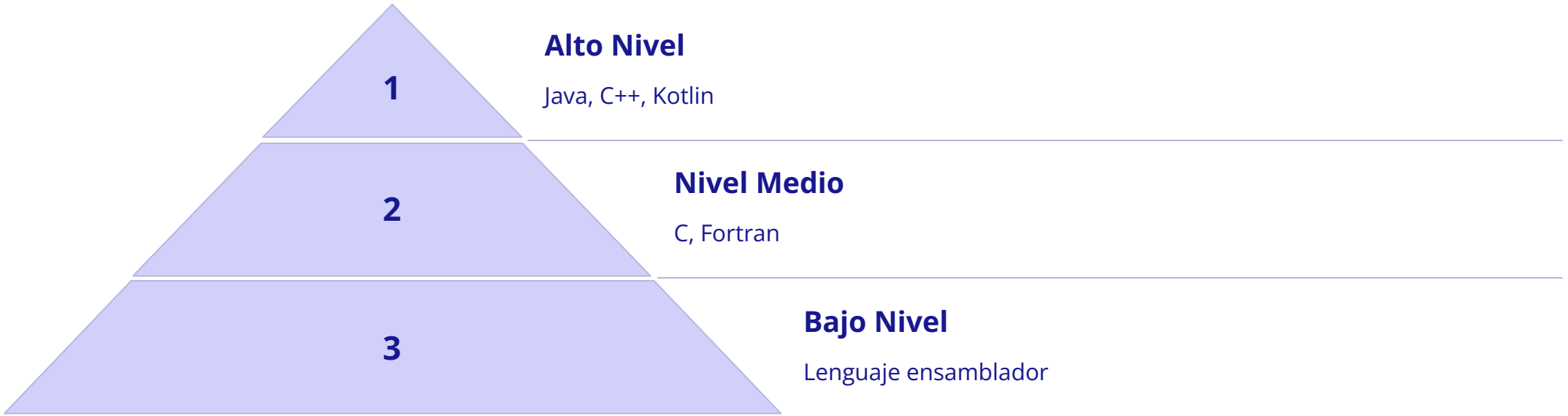
   JVM para Java, PVM para Python interpretan el bytecode

4. **Lenguaje Máquina**

   La máquina virtual compila a código binario (1 y 0) que la CPU ejecuta

Niveles de Abstracción

Además de los lenguajes de alto nivel, existen lenguajes que se acercan más a las características del hardware sobre el que se ejecutan.



Cuanto más bajo es el nivel, más control ofrece sobre el hardware, como el control sobre los registros. En consecuencia, son menos legibles.

```
MOV AX, 5  ; Guardar el número 5 en el registro AX
MOV BX, 3  ; Guardar el número 3 en el registro BX
ADD AX, BX ; Sumar BX a AX (AX ahora vale 8)
```

El **ensamblador** es un programa que traduce el lenguaje ensamblador a lenguaje máquina (1 y 0) que la CPU puede ejecutar.

/ 3. Paradigmas de programación

Una de las formas más utilizadas para la clasificación de los lenguajes de programación es el paradigma de programación utilizado. Se suelen diferenciar dos grandes grupos:

Lenguajes Imperativos

Permiten crear algoritmos a través del conjunto de instrucciones y expresiones del lenguaje.

Dices qué hacer paso a paso

- Python
- Java
- C

Lenguajes Declarativos

Solo especifican los valores que se esperan al final, sin especificar qué realizar para llegar al resultado. Es necesario indicar las relaciones lógicas y matemáticas para poder llegar al resultado.

Dices qué quieres obtener

- SQL
- HTML

Evolución de los Paradigmas

Tradicionalmente, y debido a la forma de programar del ser humano, los lenguajes imperativos son más utilizados. Dentro de los lenguajes imperativos podemos diferenciar subparadigmas como la programación orientada a objetos, programación procedimental, etc.

Paradigma Imperativo

- Programación orientada a objetos
- Programación procedimental

Paradigma Declarativo

- Programación funcional
- Programación lógica

En la actualidad, muchos lenguajes imperativos incorporan funcionalidades de la programación funcional. Por ejemplo, Kotlin y Swift utilizan funciones lambda, que son funciones de orden superior en programación funcional.

/ 4. Fases del proceso de compilación

En este apartado nos centraremos en analizar las diferentes fases por las que pasa un código fuente hasta generar el código objeto y posteriormente al código ejecutable.

Código Fuente	Código Objeto	Código Ejecutable
Lo que hemos escrito en el lenguaje de programación	Resultado de finalizar el proceso de compilación	Resultado de enlazar el código objeto con las librerías necesarias

El código ejecutable es el único que puede 'entrar' en el procesador para ser ejecutado.

Metáfora de la receta



Código Fuente

Lo que escribes tú, el chef, explicando los pasos principales de tu plato



Código Objeto

Los ingredientes ya preparados: has picado, cocido o marinado todo, listo para unirlo



Librerías

Recetas externas ya hechas que necesitas consultar, como una nota diciendo: "usa la receta 5 del libro de salsas"



Código Ejecutable

El plato servido en la mesa, listo para comer. El cliente puede comerse el plato, es decir, la CPU puede ejecutar el programa

Lenguajes compilados vs interpretados

- **Compilado:** preparar toda la receta de golpe y luego cocinarla → rápido. C++
- **Interpretado:** leer la receta paso a paso mientras cocinas → más lento, pero flexible. Python
- **Ambos:** casos especiales como Java. Código objeto → bytecode: compilado; bytecode → lenguaje máquina

Ejemplo: programa de 100 líneas con un error en la última. C++ VS Python

Fases del Compilador

Para llegar al código ejecutable el compilador debe realizar las siguientes etapas:

Análisis Léxico

Comprobar que los elementos utilizados pertenecen al lenguaje de programación

Análisis Sintáctico

Determinar si los elementos vienen en el orden correcto.

`int a = 5;`

VS

`5 = a int;`

Análisis Semántico

Determinar si las sentencias escritas tienen sentido.

`int a;`

`a = "hola";`

Metáfora del idioma

- **Análisis léxico:** Comprobar que cada letra es del idioma español
- **Análisis sintáctico:** Verificar el orden correcto: "Hola, me llamo Sergio" vs "Hola, llamo Sergio me" o
- **Análisis semántico:** En una redacción, escribir "como he mencionado antes, la tarta era de chocolate" sin haber mencionado en ningún momento ninguna tarta.



Fases del Compilador

Generación de código intermedio

Se genera una representación intermedia (bytecode) independiente del procesador. Esta fase es opcional aunque muy habitual en determinados lenguajes como Java y Python.

Optimización de código intermedio

Fase opcional cuyo objetivo es mejorar el código generado para tener un mejor rendimiento.

Generación de código final

Se genera código objeto que dependerá del conjunto de instrucciones de la CPU utilizada y se puede proceder con la generación del código ejecutable.

¿Dudas?

